Cloud DevOps / SRE Interview Q&A; Reference

Infrastructure · Networking · Security · CI/CD · AWS & Azure

1 · Infrastructure as Code (IaC)

Q1. You refactor Terraform code for existing AWS resources. Upon running terraform apply, it attempts to recreate those resources. How do you prevent this?

The problem is state drift — Terraform doesn't know your refactored code refers to the *same* resource. Three approaches:

- **terraform state mv** — rename/move resources in state to match your new resource addresses. This is the primary fix.
- **import block** (Terraform 1.5+) — declarative import directly in HCL, cleaner than the old terraform import CLI command.
- **removed block** — if removing a resource from config but not wanting it destroyed, this tells Terraform to forget it without deleting.

> *Always run terraform plan before apply after any refactor. If you see unexpected destroys, stop — don't assume it's fine.*

Q2. What specifically does Terragrunt provide that isn't native to Terraform?

- **DRY remote state config** — define your backend once, Terragrunt generates it per module. Native Terraform forces repetition.
- **Cross-module dependency graph** — dependency blocks let module B wait on module A's outputs without manual wiring.
- **run-all commands** — apply/plan/destroy across a directory tree in dependency order. No native Terraform equivalent.
- **Environment-specific variable inheritance** — inputs cascade from root → env → module level.

> *Caveat: Terragrunt adds its own complexity layer. Without strong Terraform discipline, it often makes debugging harder, not easier.*

Q3. What is a significant disadvantage of using Terraform in a large-scale environment?

State file contention. A single terraform.tfstate per workspace is a bottleneck — only one person/pipeline can hold the state lock at a time:

- Long-running pipelines block others
- Teams fight over blast radius boundaries
- State corruption risk if DynamoDB locking is misconfigured

Secondary: provider version drift across dozens of modules, and no native module registry enforcement — teams vendor their own copies, consistency breaks.

2 · Networking & Traffic Management

Q4. Why can an NLB handle millions of requests while an ALB hits scaling limits much earlier?

NLB operates at **Layer 4** (TCP/UDP). It doesn't inspect packet contents — it routes based on IP and port only:

- No SSL termination overhead (unless explicitly configured)
- No HTTP parsing, header inspection, or routing logic
- Connections handled at the kernel network stack level

ALB operates at **Layer 7** — it parses every HTTP request, evaluates rules, handles TLS termination, inspects headers, manages WebSocket upgrades. Each connection carries more compute cost.

NLB also preserves client IP natively and has static IP addresses per AZ — useful for whitelisting.

Q5. Trace the path: When a client hits a URL, explain the step-by-step journey until it reaches an EC2 instance.

1. **DNS resolution** — browser queries recursive resolver → authoritative DNS (Route 53) → returns IP or CNAME chain
2. **TCP handshake** — client opens connection to resolved IP on port 443
3. **TLS handshake** — certificate exchange, cipher negotiation, session key established
4. **HTTP request sent** over encrypted tunnel
5. **Load balancer receives request** — ALB/NLB in VPC (sits in public subnet)
6. **Security Group on LB evaluated** — allows 443 inbound
7. **Target Group lookup** — ALB selects EC2 target based on routing rules + health check status
8. **ALB forwards to EC2** — new TCP connection from LB to EC2 private IP on target port
9. **Security Group on EC2 evaluated** — must allow inbound from LB SG (not 0.0.0.0/0)
10. **Application receives request** on the EC2 instance

> *Common gap: EC2 SG should reference the LB's security group ID, not be open to the world.*

Q6. Transit Gateway vs. VPC Peering — what's the difference?

	VPC Peering	Transit Gateway
Topology	1-to-1	Hub-and-spoke (many-to-many)
Transitive routing	Not supported	Supported
Cross-account	Yes	Yes
Cost	Data transfer only	Per attachment + data transfer
Routing at scale	Explodes (N^2 connections)	Managed centrally

Key point: VPC Peering doesn't allow transitive routing. A→B and B→C does *not* mean A can reach C through B. Transit Gateway solves this. At 3+ VPCs, TGW almost always wins on operational simplicity despite the cost.

3 · Security, CI/CD & Docker

Q7. TLS vs. SSL — actual differences, and how to configure, store, and rotate certificates?

SSL is deprecated. SSLv2 was broken in the 90s; SSLv3 broken by POODLE in 2014. Nobody should be running SSL. TLS 1.0 and 1.1 are also deprecated (RFC 8996). Enforce **TLS 1.2 minimum**, TLS 1.3 preferred.

Storage:

- ACM for AWS-managed certs (auto-renewal)
- Secrets Manager / Parameter Store (SecureString) for self-managed certs. Never flat files on EC2.

Rotation:

- ACM handles this automatically for certs it issues.
- For self-managed: automate with Certbot + Lambda, triggered by EventBridge rule before expiry.
- Set CloudWatch alarms on cert expiration days-remaining metrics.

> *Common gap: teams configure cert rotation but never test that the new cert is picked up by the application without a restart.*

Q8. How do you prevent unverified images from being pulled from a Docker registry?

Layered approach:

- **Docker Content Trust (DCT)** — enforces image signing via Notary. Set DOCKER_CONTENT_TRUST=1.
- **Private registry with pull-through cache** — ECR or Artifactory proxying Docker Hub. Control what's cached; scan before allowing.
- **Image scanning in CI** — Trivy, Snyk, or ECR native scanning on every build. Fail the pipeline on HIGH/CRITICAL CVEs.
- **OPA/Gatekeeper in Kubernetes** — admission controller that rejects pods referencing images not from approved registries.
- **Digest pinning** — reference images by SHA256 digest, not tags. Tags are mutable; digests are not.

> *Weakest link: teams scan in CI but don't prevent runtime pulls of un-scanned images. The admission controller closes that gap.*

Q9. Which SAST and DAST tools have you implemented in CI/CD pipelines?

SAST: Semgrep, Checkmarx, Bandit (Python), SonarQube, tfsec/Checkov (IaC-specific)

DAST: OWASP ZAP, Burp Suite Enterprise, Nuclei for templated scanning

Secrets scanning: GitLeaks, truffleHog, GitHub Advanced Security — run on every commit pre-merge

> *Most teams implement SAST and treat DAST as optional. DAST is harder to automate without false positives, but skipping it means you're not testing running application behavior.*

Q10. Explain the high-level architecture of a production CI/CD infrastructure.

```
Code Push → SCM (GitHub/GitLab)
→ Webhook triggers pipeline (GitHub Actions / GitLab CI / Jenkins)

Pipeline stages:
1. Lint + unit tests
2. SAST + secrets scan
3. Docker build → image scan (Trivy)
4. Push to private registry (ECR) with digest
5. Deploy to staging (Helm/Terraform)
```

```
6. Integration + DAST tests
7. Manual approval gate (for prod)
8. Deploy to prod (blue/green or canary)
9. Smoke tests + rollback trigger on failure
```

What separates mature pipelines: the rollback is automatic and tested, not a runbook someone follows at 2am.

4 · Cloud Specifics (AWS & Azure)

Q11. What are the various ways to cache S3 bucket content for better performance?

- **CloudFront** — primary answer. CDN with edge caching, TTL control, cache behaviors per path, Origin Shield for additional cache layer.
- **Browser caching** — set Cache-Control headers on S3 objects (max-age, s-maxage).
- **ElastiCache** — if your application fetches S3 metadata or small objects repeatedly, cache at app layer.
- **CloudFront + Lambda@Edge** — for dynamic cache key manipulation or auth at edge.
- **S3 Transfer Acceleration** — speeds up *writes to S3* via edge locations, not reads. Not a caching solution.

People often conflate Transfer Acceleration with read performance. CloudFront is what you want for read/delivery performance.

Q12. What is a Resource Group in Azure, and what is its primary purpose?

A **logical container** for Azure resources that share the same lifecycle. Primary purposes:

- **Unified lifecycle management** — delete the RG, everything in it is deleted. Useful for ephemeral environments.
- **RBAC scope** — assign permissions at the RG level instead of per-resource.
- **Cost tracking** — filter billing by RG.
- **Policy assignment** — Azure Policy can be scoped to a RG.

Key constraint: resources in a RG can span regions. The RG 'location' is only for its metadata — resources inside can be anywhere. This trips up people who assume RG = single-region boundary.

The third leg most SRE/DevOps work skips: **blast radius thinking** — not just *how* things work, but *what fails, how loudly, and how you recover*. That's where interview answers and production systems alike tend to fall apart.